AUTOMATED REVIEW RATING SYSTEM

PROJECT OVERVIEW

The Automated Review System processes and analyzes customer reviews to predict ratings or sentiment automatically. It cleans and preprocesses review data, splits it for training and testing to ensure reliable evaluation, and uses machine learning models to classify reviews. This system provides businesses with fast, data-driven insights into customer satisfaction, product quality, and areas for improvement, enabling better decision-making and enhanced customer experience.

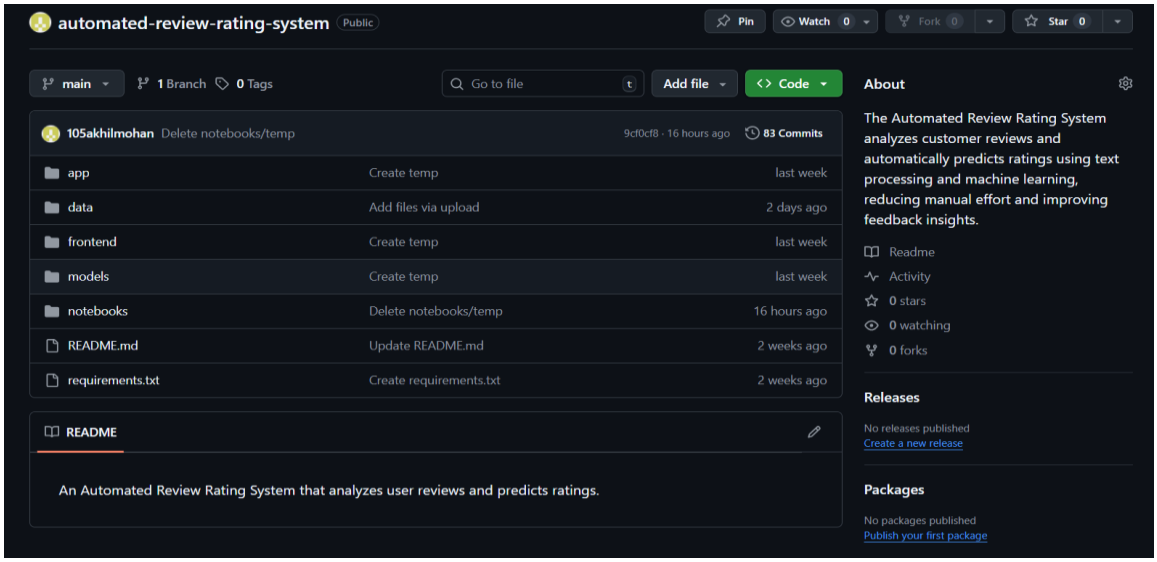
ENVIRONMENTAL SETUP

The project is executed in Google Colab, which provides a cloud-based Python environment with free CPU and GPU resources. The setup includes:

- Python Version: Python 3.x (pre-installed in Colab)
- Data Handling & Preprocessing: pandas, numpy
- Text Processing & NLP: scikit-learn (for train-test split, feature extraction, and machine learning models)
- Visualization: matplotlib, seaborn (for plotting data distributions and results)

GITHUB SETUP

The project was set up on GitHub, a new repository was created to store datasets, preprocessing scripts, and model files. The setup process included initializing the repository, configuring Git on the local system, and pushing the project files to GitHub. A screenshot of the GitHub repository setup is included below to demonstrate the successful configuration and repository structure for the Automated Review Rating System.



DATA COLLECTIONS

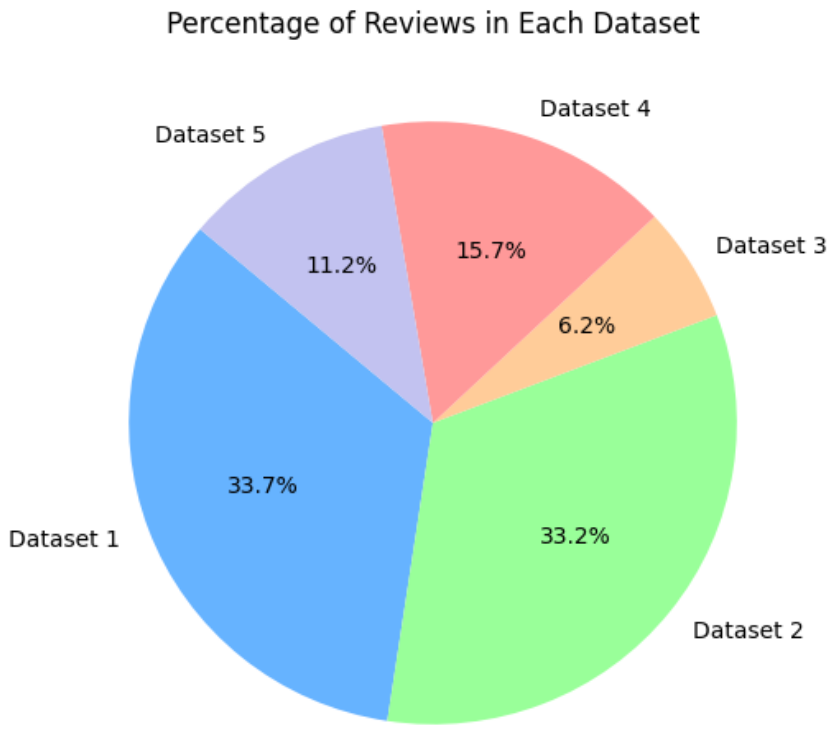
The dataset for this project was sourced from several publicly available Kaggle repositories, encompassing a diverse range of reviews for sentiment analysis:

- Amazon Reviews for Sentiment Analysis: A collection of Amazon product reviews in fastText format, facilitating sentiment classification tasks. kaggle kaggle
- ChatGPT Users' Reviews: User feedback on ChatGPT, including textual comments, ratings, and review dates, offering insights into user experiences. kaggle
- Sentiment Analysis Dataset: A dataset containing labeled sentiment data, suitable for training and evaluating sentiment analysis models. kaggle
- Reviews Dataset: A compilation of reviews from various industries, providing a broad spectrum of textual data for analysis. kaggle

The collected datasets, including Amazon product reviews, ChatGPT user reviews, and various sentiment-labeled review datasets, have been uploaded and made publicly available on GitHub for reproducibility and ease of access. github

Pie-chart

A pie chart was created to visualize the distribution of reviews across five different datasets. The chart shows the relative percentage of reviews contributed by each dataset, highlighting their proportion in the combined corpus



DATA PREPROCESSING

In this stage, several cleaning and transformation steps were performed to ensure data quality and consistency.

Handling Missing Data

Missing values in the dataset were identified and appropriately handled to maintain data consistency and avoid bias during model training.

```
df = df.dropna(subset=["userName", "content", "score"])

df["thumbsUpCount"] = df["thumbsUpCount"].fillna(0)
```

Removing Duplicates

Duplicate entries were removed to ensure that repeated reviews did not distort the analysis or affect the accuracy of the system.

```
df = df.drop_duplicates()
```

Feature Engineering

New features such as review length and sentiment indicators were created to enhance the dataset's representation and improve model performance.

```
df["review_length"] = df["content"].apply(lambda x: len(str(x).split()))
```

Removing Very Short Reviews

Reviews containing fewer than three words were removed, as they carry little semantic meaning and do not contribute effectively to the analysis.

```
df = df[df['Review'].apply(lambda x: len(str(x).split()) > 4)]
```

Removing Very Long Reviews

Reviews exceeding 100 words were removed from the dataset to ensure consistency and maintain manageable input lengths for analysis. This helps improve processing efficiency and model performance by focusing on concise, meaningful reviews.

```
df = df[df['review'].apply(word_count) <= 100]
```

Drop Blank Reviews

Blank reviews, which contain no meaningful text, were identified and removed from the dataset during preprocessing. This step ensures that only valid and informative reviews are retained for analysis and model training.

```
df = df[df['Review'].notna() & (df['Review'].str.strip() != '')]
```

Removing Special Characters and Emojis

Unnecessary special characters and emojis were eliminated from the text to standardize the review content and make it more suitable for text processing.

```
text = re.sub(r'[^A-Za-z0-9 ]+', '', text)
```

Remove conflicting reviews

Conflicting reviews, where the same user provided multiple reviews with different ratings for the same product, were identified and removed. This step ensures consistency in the dataset and prevents misleading information from affecting the training and evaluation of the model.

```
conflicts = df.groupby('Review')['Rating'].nunique()

conflict_reviews = conflicts[conflicts > 1].index

df = df[~df['Review'].isin(conflict_reviews)]
```

Removing URLs

URLs present in reviews were removed since they do not contribute to sentiment or review quality and could introduce noise into the dataset.

```
df['review'] = df['review'].str.replace(r'http[s]?://\S+|www\.\S+', '', regex=True)
```

Remove other language

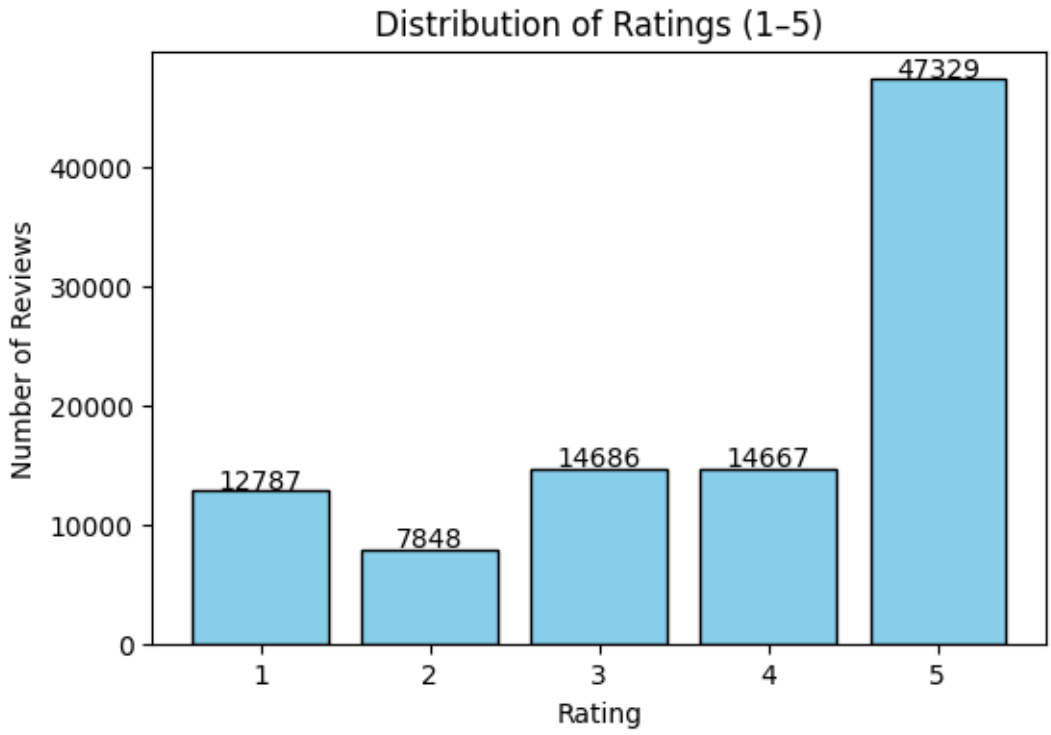
To maintain consistency and improve model performance, reviews in languages other than English are removed. This ensures that the dataset contains only English text for preprocessing and training.

```
english_df = df[df["language"] == "en"].drop(columns=["language"]).reset_index(drop=True)
```

The cleaned and preprocessed dataset, ready for training and analysis, has been uploaded to GitHub for easy access. It includes all reviews with duplicates, very short entries, URLs, special characters, and emojis removed, along with added features like review length. The dataset can be accessed here. github

DATA VISUALIZATION

After preprocessing, a bar chart was generated to display the distribution of reviews across different rating categories. This visualization helps in understanding the balance of the dataset and ensures that preprocessing steps such as removing duplicates, short reviews, and special characters did not distort the class proportions.



CONVERTING TO LOWERCASE

All review text was converted to lowercase to ensure uniformity and reduce duplication caused by capitalization differences.

df = df.applymap(lambda s: s.lower() if isinstance(s, str) else s)

IMBALANCED DATASET

The imbalanced dataset was created with a total of **10,000 samples** distributed unequally across rating categories. Specifically, it contains 1,200 samples of rating 1 (12%), 1,000 samples of rating 2 (10%), 2,800 samples of rating 3 (28%), 2,000 samples of rating 4 (20%), and 3,000 samples of rating 5 (30%). These proportions highlight class imbalance, where certain ratings are more frequent, closely resembling real-world review distributions.

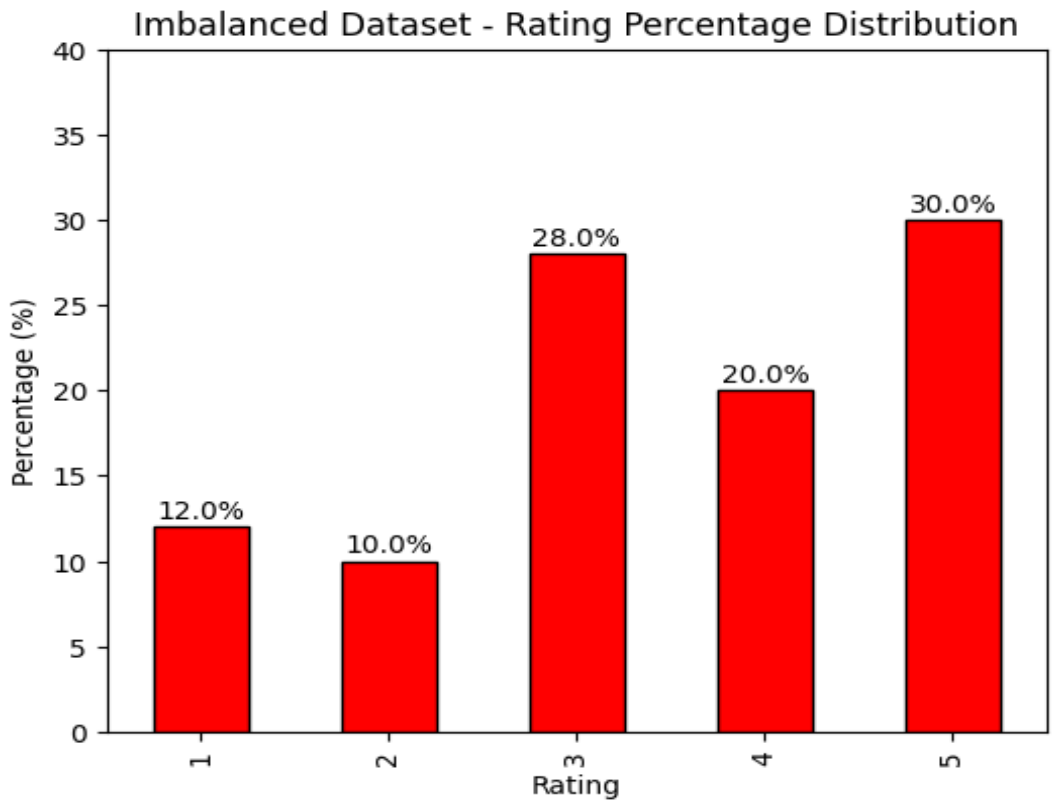
Code

```
imbalanced_counts = {
    1: 1200,
    2: 1000,
    3: 2800,
    4: 2000,
    5: 3000
```

```
}

# CREATE IMBALANCED DATASET
imbalanced_parts = []
remaining_parts = []

for rating, count in imbalanced_counts.items():
    rating_df = df[df['rating'] == rating]
    imbalanced_sample = rating_df.sample(n=count, random_state=42)

    imbalanced_parts.append(imbalanced_sample)

    # Remove chosen samples so they are not reused
    remaining = rating_df.drop(imbalanced_sample.index)
    remaining_parts.append(remaining)

imbalanced_df=pd.concat(imbalanced_parts).sample(frac=1,
random_state=42).reset_index(drop=True)
```

DATA VISUALIZATION

The imbalanced dataset was visualized using a bar graph to show the distribution of samples across all ratings. The graph highlights that rating 5 (30%) and rating 3 (28%) are the most frequent, followed by rating 4 (20%), while rating 1 (12%) and rating 2 (10%) are underrepresented. This visualization effectively demonstrates the imbalance, making it clear which ratings dominate the dataset compared to others.



SAMPLES OF 1 STAR RATING

Review 1:

worst ever support you will receive from phonepe support team whatever you ask they dont have answer and supervisor is always busy in handling escalated calls and do not worry about customer issues their attitude will be like you used phonepe app and it is your headache now

Review 2:

it was best app with more options to interact but i can’t see pollsthe dont open it flashes only white screen i updated the newest version still can’t vote on polls or can’t create one even many of my friends can’t open polls note using samsung galaxy m30s with 6gb ram and 128gb internal memory fix it to except 5 star

Review 3:

forces me to purchase the full version without having choice to use free version scam 0 stars

Review 4:

am i the only one who is not seeing the voice interaction button have they removed it

Review 5:

i tried to log in into the app but i tried continuously 20 times and the login in goes infinity fix this problem

SAMPLES OF 2 STAR RATING

Review 1:

i have used this app before and it was surely great but i havent used it for a while and now that i am using it again i hate it because even though my signal is okay i cannot seem to connect to other people hope you fix your problems sooner

Review 2:

file tree default apps selection internal storage shown as external its not rocket science more options is more better get it done

Review 3:

my bank account not added i trying so much time but error shown please solve my issue asapthank you so much

Review 4:

not helping with more options its just ok ok

Review 5:

want to go back its slow will not pick up my sd card anymore it sucks i have paid version about to bail out on it

SAMPLES OF 3 STAR RATING

Review 1:

its a nice app its fast but the reason i gave it 3 stars its because of ads it has too much ads

Review 2:

as of today 23-01-2021 need to quickly improve as per listed 1 unable to zoom in received photo please improve to enable 2 dont have live location sharing please provide live location sharing 3 unable to viewzoom other contact profile picture 4 please add in sticker application thank you congratulation for your good initiative

Review 3:

this aap is nice but now its not working properly i want to move my pictures and videos in sd card but i cant move i hope you can help in this so plz tell what should i do

Review 4:

but only need to be better

Review 5:

i am not receiving notifications from telegram i have checked all notification settings in phone and also telegram every thing seems fine but i am still not getting any notifications in notifications bar any resolution for this

SAMPLES OF 4 STAR RATING

Review 1:

it sometime gives the inaccurate information

Review 2:

its crazy good just gives me some irresponsible answers sometimes

Review 3:

good but need update hire engineers from gptc mndy wayanad kerala to improve

Review 4:

overall a good app for small bakery and shops but only one concernfull thing is that many times the barcode doesnt work and payments get failed overall i loved this app

Review 5:

there should be a paid version of chatgpt that isnt restricted in anyway for example just to test it i asked how to torrent files and it denied me things shouldnt be restricted

SAMPLES OF 5 STAR RATING

Review 1:

the best and the fastest answering app

Review 2:

i recently downloaded this app and its astonishing it literally provides answers before you can think its 10 times faster than google takes you to your answer right away this was being trending and that i have chosen to give it an up

Review 3:

i have just downloaded chatgpt and so far i am quite impressed by what it can do the response to questions that i have asked it it has provided descriptive and informative answers i still have much to learn and to experience and i will be able to review at a later time with more insight and detail

Review 4:

very usefull and informative app

Review 5:

im very impressed that it is very good at understanding natural language and replies are almost fast enough hoping that there will be a multi modal ai feature in the future updates so that i could get help from real time scenarios

STOPWORD REMOVAL

After creating the imbalanced dataset, stopword removal was applied to clean the review text. Stopwords are common words (like "the", "is", "and") that do not carry significant meaning for

sentiment or rating analysis. Negations such as “no”, “not”, and “never” were retained to preserve sentiment context. This step helps reduce noise in the dataset and improves the effectiveness of subsequent feature extraction and model training.

SpaCy is used in the preprocessing pipeline to efficiently tokenize and lemmatize the text data. By leveraging nlp.pipe, the system processes large batches of reviews quickly while removing non-alphabetic characters and stopwords, preserving meaningful words and negations for accurate sentiment analysis.

Code

```
# Load spaCy model

nlp = spacy.load("en_core_web_sm", disable=["parser", "ner", "textcat"])

# Define custom stop words (keep negations)

stop_words = set(stopwords.words('english'))

negations = {"no", "not", "nor", "never", "n't"}

stop_words = stop_words – negations
```

Removed words

During preprocessing, stopwords common words like “the,” “and,” and “is” that carry little semantic meaning were identified in the dataset. After excluding negations such as “not” and “never,” a total of 129 unique stopwords were found across all reviews. Removing these stopwords helps in focusing on meaningful words, improving the efficiency and accuracy of text analysis.

Stopwords

['a', 'about', 'above', 'after', 'again', 'against', 'all', 'am', 'an', 'and', 'any', 'are', 'as', 'at', 'be', 'because', 'been', 'before', 'being', 'below', 'between', 'both', 'but', 'by', 'can', 'd', 'did', 'didn', 'do', 'does', 'doing', 'don', 'down', 'during', 'each', 'few', 'for', 'from', 'further', 'had', 'has', 'have', 'having', 'he', 'her', 'here', 'him', 'himself', 'his', 'how', 'i', 'if', 'in', 'into', 'is', 'it', 'its', 'itself', 'just', 'm', 'ma', 'me', 'more', 'most', 'my', 'myself', 'now', 'o', 'of', 'off', 'on', 'once', 'only', 'or', 'other', 'our', 'ourselves', 'out', 'over', 'own', 're', 's', 'same', 'she', 'should', 'so', 'some', 'such', 't', 'than', 'that', 'the', 'their', 'theirs', 'them', 'themselves', 'then', 'there', 'these', 'they', 'this', 'those', 'through', 'to', 'too', 'under', 'until', 'up', 've', 'very', 'was', 'we', 'were', 'what', 'when', 'where', 'which', 'while', 'who', 'whom', 'why', 'will', 'with', 'won', 'y', 'you', 'your', 'yours', 'yourself']

Why is stemming preferred over lemmatization?

Stemming is often preferred over lemmatization when speed is more important than linguistic accuracy. Unlike lemmatization, which converts words to their correct dictionary forms using vocabulary and grammar rules, stemming simply chops off word endings to reduce words to a root form. This makes stemming faster and less resource-intensive, especially for large

datasets, though it may produce non-standard or incomplete words. It is useful in applications like search engines or text classification where exact word forms are less critical.

Lemmatization

Lemmatization is a text preprocessing technique used to reduce words to their base or dictionary form, called a lemma. Unlike stemming, which may produce incomplete or non-dictionary words, lemmatization preserves the actual meaning of words. This is particularly useful in sentiment analysis and review processing, as it ensures that different forms of a word are treated consistently. For example, the words “running”, “ran”, and “runs” are all converted to “run”, and “better” is converted to “good”. By applying lemmatization, the model can better understand the semantic content of the reviews.

Code

```
def preprocess_doc(doc):
    tokens = [
        token.lemma_ for token in doc
        if token.is_alpha and token.text not in stop_words
    ]
    return " ".join(tokens)
```

SPLITTING THE IMBALANCED DATASET

The imbalanced dataset of 10,000 samples was split into 80% training (8,000 samples) and 20% testing (2,000 samples) while preserving the same class proportions. In both train and test sets, the distribution remains consistent: rating 1 (12%), rating 2 (10%), rating 3 (28%), rating 4 (20%), and rating 5 (30%). This ensures that the imbalance is reflected equally in both subsets, maintaining representativeness for model training and evaluation.

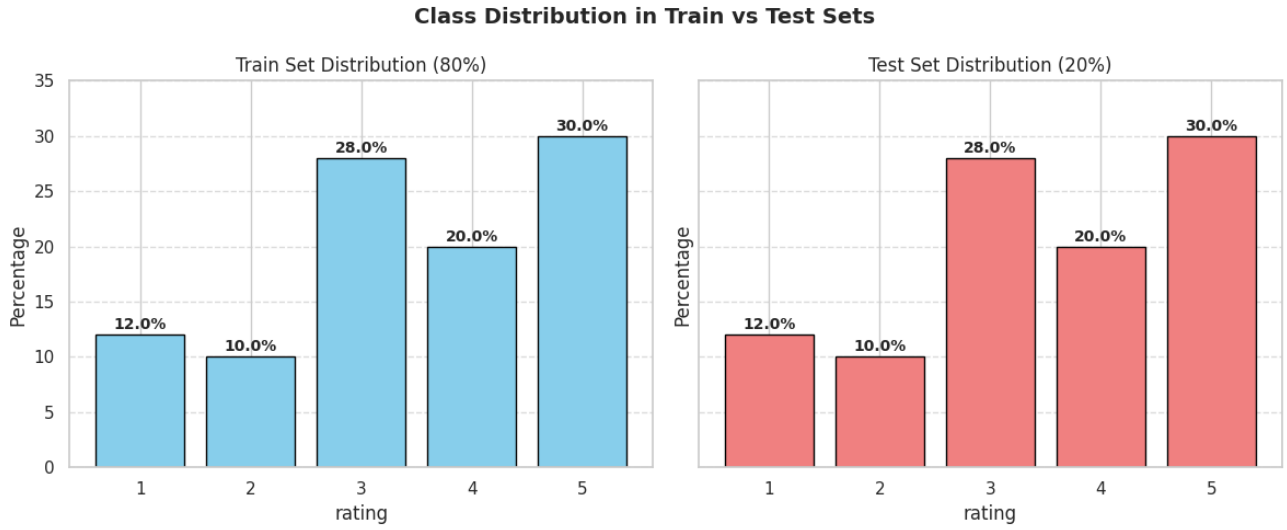
Code

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

The preprocessed train (80%) and test (20%) datasets are available on GitHub for access. github

DATA VISUALIZATION

To verify the distribution of reviews after splitting, a bar chart was plotted showing the count of each rating in the training and testing sets. The chart confirms that both sets maintain a proportional distribution of ratings, thanks to stratified splitting.



TF-IDF CALCULATION

Term Frequency–Inverse Document Frequency (TF-IDF) is a numerical statistic that reflects how important a word is to a document relative to a collection of documents (corpus). It helps highlight significant words while reducing the weight of common words. TF-IDF is calculated as:

$$\mathrm{TF(t,d)} = \left(\frac{\text{Number of times term t appears in document d}}{\text{Total number of terms in document d}}\right)$$

$$IDF(t) = log\left(\frac{Total\ number\ of\ documents}{Number\ of\ documents\ containing\ term\ t}\right)$$

Code

```
# Initialize TF-IDF

tfidf = TfidfVectorizer(max_features=5000, stop_words='english')

# Fit only on training data

X_train_tfidf = tfidf.fit_transform(train_df['review'])

# Transform test data using the same vocabulary and IDF

X_test_tfidf = tfidf.transform(test_df['review'])
```

MODEL PERFORMANCE ON TRAINING

The evaluation of multiple machine learning models trained on a imbalanced text review dataset to predict sentiment or rating categories. The balanced dataset ensures equal representation of all classes, allowing a fair comparison of model performance without bias toward dominant classes. Five models are Logistic Regression, Naive Bayes, Decision Tree, Random Forest, and SVM were assessed using key metrics such as accuracy, precision, recall, and F1-score to determine their effectiveness and generalization capability for text classification tasks.

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.5230	0.4940	0.5275	0.4880
Decision Tree	0.4105	0.3991	0.4105	0.4039
Random Forest	0.5015	0.4332	0.5015	0.4223
SVM	0.5275	0.4911	0.5230	0.4849
Naïve Bayes	0.4980	0.5170	0.4980	0.3850

Among all models trained on the balanced dataset, SVM achieved the highest overall performance with an accuracy of 0.5275, followed closely by Logistic Regression at 0.5230. Both models demonstrated strong precision-recall indicating less balance prediction, indicating effective generalization across all rating classes. Tree-based models like Decision Tree and Random Forest showed lower performance, suggesting that linear models are better suited for the TF-IDF-based text representation used in this dataset.

Fine-Tuning Results on Balanced Dataset

The fine-tuning experiments on the imbalanced dataset show that SVM with an RBF kernel (C=1) achieved the highest accuracy of 0.5380, outperforming all other models. Logistic Regression performed well with C=1 (0.5295), while Random Forest and Decision Tree showed moderate improvements with increasing estimators and depth, respectively. Naïve Bayes performed reasonably but lagged behind SVM and Logistic Regression. Overall, SVM proved most effective for handling imbalanced class distributions after hyperparameter tuning.

Model	Hyperparameter	Accuracy	Best Parameters
Logistic Regression	C = 0.01 - 0.4745 C = 0.1 - 0.5060 C = 1 - 0.5295 C = 10 - 0.4880	0.5295	C = 1
SVM	kernel = linear - 0.5215 kernel = poly - 0.4855 kernel = rbf - 0.5380 kernel = sigmoid - 0.5160	0.5380	kernel = rbf, C = 1

Decision Tree	max_depth = 5 - 0.4315 max_depth = 10 - 0.4300 max_depth = 20 - 0.4305 max_depth = None - 0.3975	0.4340	max_depth = 20, criterion = 'gini'
Random Forest	n_estimators = 50 - 0.4990 n_estimators = 100 - 0.5115 n_estimators = 200 - 0.5180	0.5230	n_estimators = 200, max_depth = None
Naïve Bayes	alpha = 0.1 - 0.5065 alpha = 0.5 - 0.5080 alpha = 1.0 - 0.5015 alpha = 2.0 - 0.4950	0.5080	alpha = 0.5

The SVM model achieved the highest accuracy after fine-tuning, demonstrating its superior performance on the balanced dataset.

Cross-Test Performance (Imbalanced - Balanced)

When the model trained on the imbalanced dataset was tested on the balanced dataset, it achieved a **cross-test accuracy of 0.4110**, with a **weighted precision of 0.3904** and a **weighted F1 score of 0.3622**. These results indicate that the model's performance drops when applied to a balanced distribution, suggesting that training on imbalanced data can reduce generalization to evenly distributed classes.

Which model would you deploy and why?

For the imbalanced dataset scenario, SVM with an RBF kernel (C=1) is the recommended model for deployment. It achieved the highest accuracy (0.5380) after fine-tuning and demonstrated robust handling of the skewed class distribution. While Logistic Regression was close in performance (0.5295), SVM's non-linear RBF kernel allows it to capture complex patterns in high-dimensional TF-IDF features more effectively. This makes it the most reliable choice for generalization and stable performance when predicting across uneven class distributions in production.